

GUÍA VISUAL · ELI5 → HERO

Taylor de dos variables

con diferencias finitas y Pascal

Una explicación que empieza con una montaña de plastilina, llega paso a paso a la fórmula y termina enseñando exactamente qué hace el programa.

No necesitas saber cálculo para empezar.

Python · uv · NumPy · mpmath · Matplotlib · Ruff · Pytest

Documento reproducible escrito en Typst · Septiembre de 2026

1 Antes de comenzar: el mapa completo

Imagina que alguien te da una función complicada y te pregunta:

“¿Puedes construir una versión más sencilla que se parezca a ella?”

Eso es todo lo que vamos a hacer. La versión sencilla será un **polinomio**: una suma de números, x , y , x^2 , xy , y^2 y más piezas parecidas.



La historia en una sola frase

Miramos la altura y las inclinaciones de la función cerca de un punto, estimamos esas inclinaciones dando pasitos diminutos, y mezclamos toda la información usando números del triángulo de Pascal.

1.1 Las siete estaciones del viaje

N.º	Idea	Pregunta que responde
1	Función	¿Cómo es la superficie original?
2	Centro	¿Alrededor de qué punto queremos copiarla?
3	Derivadas	¿Hacia dónde se inclina y cómo se curva?
4	Diferencias finitas	¿Cómo estimamos inclinaciones sin derivar a mano?
5	Pascal	¿Cómo repartimos cada orden entre x e y ?
6	Taylor	¿Cómo armamos el modelo sencillo?
7	Gráfica	¿Dónde coinciden el original y el modelo?

✓ Comprobación rápida

Si por ahora entiendes “vamos a copiar localmente una forma complicada con piezas sencillas”, ya tienes la idea más importante.

2 Una función de dos variables es una montaña

Una función de una variable, como $f(x) = x^2$, recibe un número y devuelve otro. Podemos dibujarla como una curva.

Una función de dos variables recibe **dos** números:

$$f(x, y) = \text{una altura}$$

Piensa en un mapa:

- x dice cuánto caminar al este o al oeste.
- y dice cuánto caminar al norte o al sur.
- $f(x, y)$ dice la altura del suelo en ese lugar.

Una función de dos variables se puede mirar como una montaña

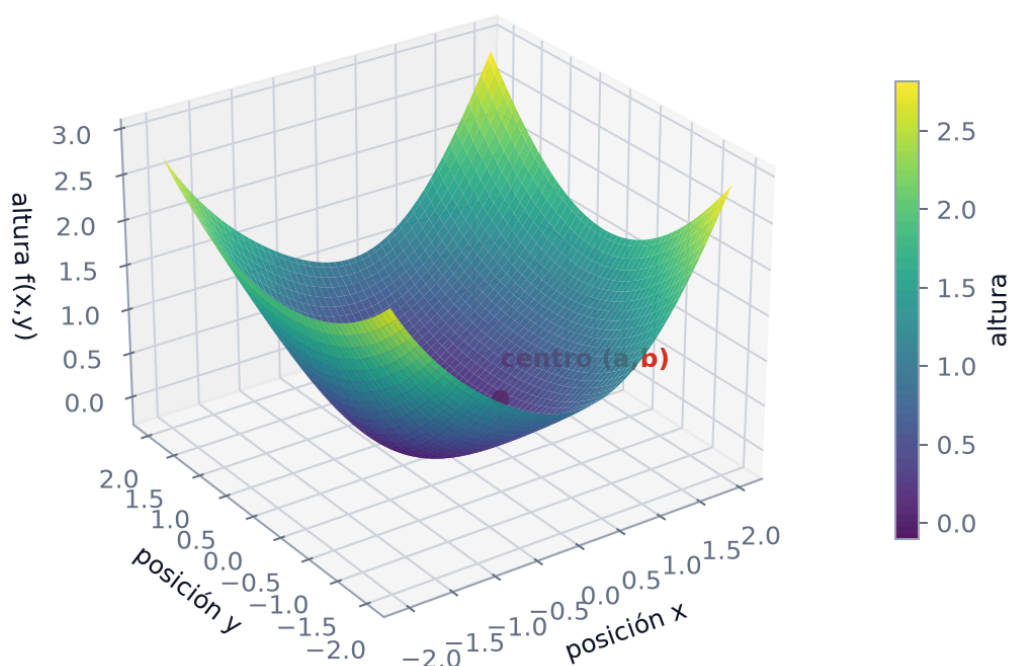
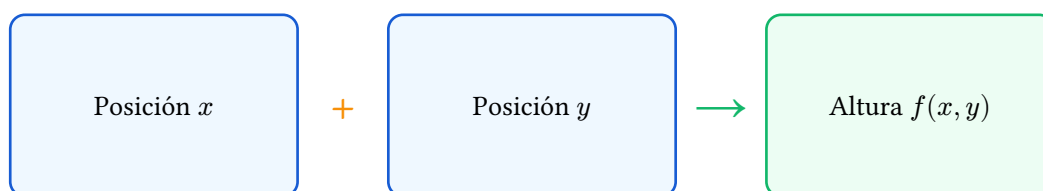


Figura 1: **Función como montaña.** Cada pareja (x, y) elige un lugar y la superficie da su altura. El punto rojo es el centro donde colocamos nuestra lupa.



2.1 Ejemplo: un tazón

$$f(x, y) = x^2 + y^2$$

En $(0, 0)$ la altura es cero. Al alejarnos en cualquier dirección, la altura crece. La superficie parece un tazón.

2.2 Ejemplo: una silla de montar

$$f(x, y) = x^2 - y^2$$

En la dirección de x sube, pero en la dirección de y baja. El mismo punto puede tener comportamientos diferentes según la dirección.

**Por qué necesitamos varias derivadas**

Una sola “inclinación” ya no basta. Necesitamos preguntar por la inclinación en x , la inclinación en y , las curvaturas y también cómo interactúan ambas direcciones.

3 El centro: nuestra lupa matemática

Taylor no intenta copiar toda la montaña de una vez. Primero elegimos un punto:

$$(a, b)$$

Este es el **centro de expansión**. Imagina que colocamos una lupa allí.

Mundo completo: quizá enorme y complicado

Zona vista por la lupa alrededor
de (a, b)

Cuanto más cerca estamos del centro, mejor suele funcionar la copia de Taylor.

Taylor es una copia local: el centro importa

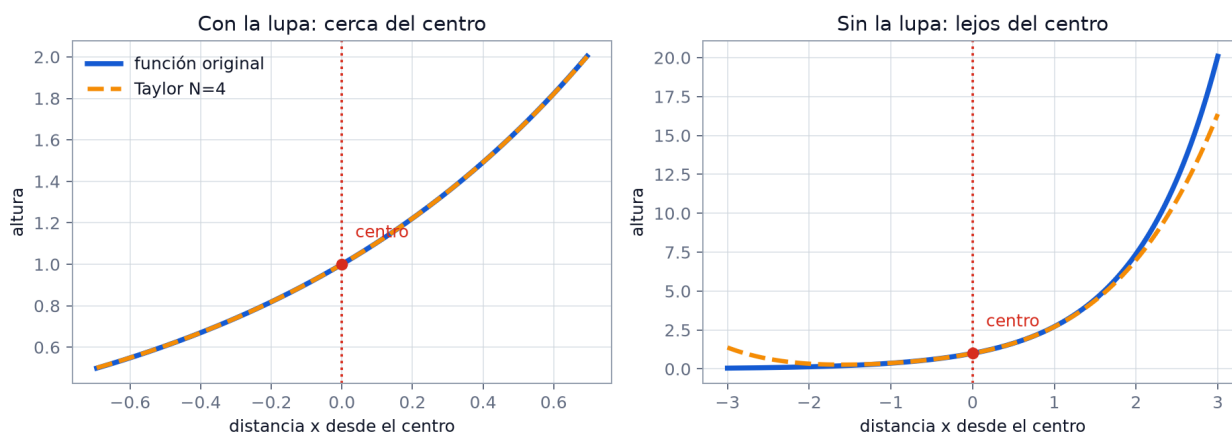


Figura 2: **La misma aproximación vista con dos escalas.** Cerca de cero las curvas se confunden; lejos del centro empiezan a separarse. No cambió el polinomio: cambió cuánto nos alejamos.

3.1 Un cambio de coordenadas muy útil

En vez de repetir $x - a$ e $y - b$ con palabras, llamamos:

$$\Delta x = x - a, \quad \Delta y = y - b$$

- Δx es cuánto nos alejamos horizontalmente del centro.
- Δy es cuánto nos alejamos verticalmente del centro.
- En el propio centro, ambos valen cero.

⚠ Local no significa global

Un Taylor excelente cerca del centro puede ser pésimo lejos de él. Si el dibujo usa un rango muy grande, ver separación entre superficies no implica automáticamente que el algoritmo esté roto.

4 Derivadas: velocímetros de la superficie

Una derivada responde: “si doy un pasito, ¿cuánto cambia la salida?”.

En una carretera:

- la posición dice dónde estás;
- la primera derivada se parece a un velocímetro;
- la segunda derivada cuenta cómo cambia ese velocímetro.

En nuestra montaña tenemos dos velocímetros principales:

$$f_{x(a,b)} \quad \text{y} \quad f_{y(a,b)}$$

f_x mide el cambio al movernos en x manteniendo y quieta. f_y hace lo mismo al movernos en y .

4.1 Órdenes de derivación

Orden total	Símbolos	Qué cuentan
0	f	La altura.
1	f_x, f_y	Las dos inclinaciones.
2	f_{xx}, f_{xy}, f_{yy}	Curvaturas e interacción.
3	$f_{xxx}, f_{xxy}, f_{xyy}, f_{yyy}$	Cómo cambian las curvaturas.

La derivada f_{xy} es **mixta**: primero pregunta por un cambio en una dirección y luego por cómo cambia eso en la otra.

4.2 ¿Cuántas derivadas aparecen hasta N?

En el orden exacto n aparecen $n + 1$ combinaciones. Hasta orden N aparecen:

$$1 + 2 + \dots + (N + 1) = \frac{(N + 1)(N + 2)}{2}$$

Con $N = 4$ son 15 términos. Con $N = 10$ son 66. Por eso reutilizar cálculos es importante.

5 Diferencias finitas: medir con pasitos

Supón que no sabemos derivar una función a mano, pero sí podemos evaluarla. Damos un paso pequeño h :

$$\text{inclinación} \approx \frac{f(a+h) - f(a)}{h}$$

Esto es simplemente:

$$\frac{\text{altura nueva} - \text{altura anterior}}{\text{distancia caminada}}$$

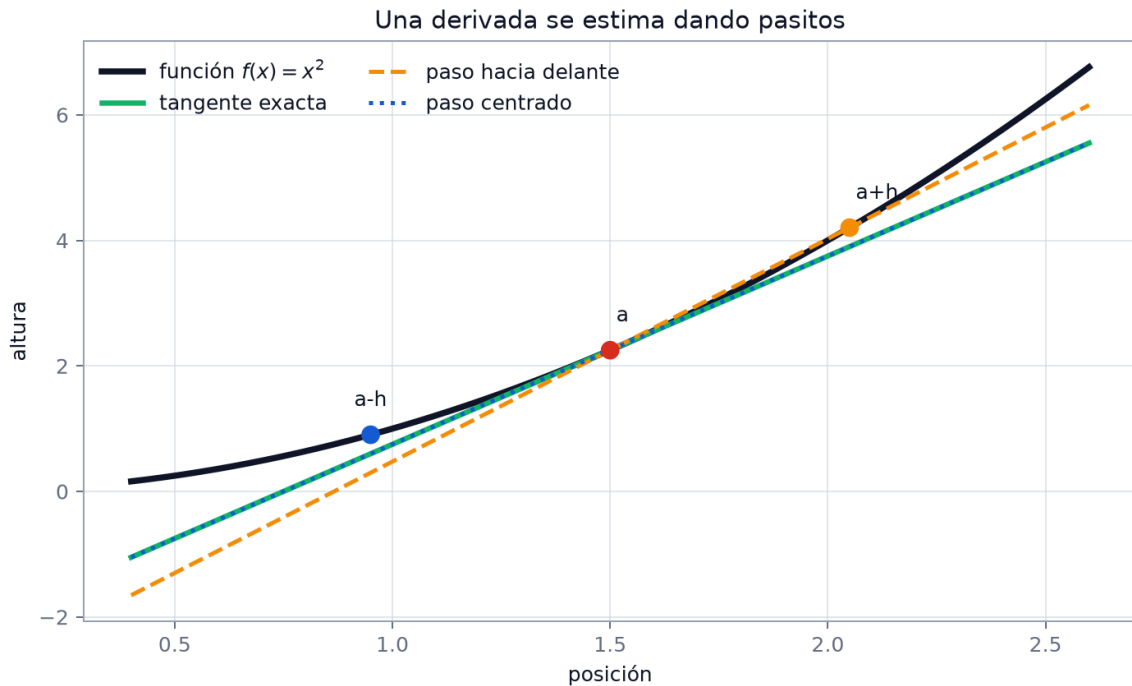


Figura 3: **Medir una pendiente con puntos cercanos.** La recta naranja mira hacia delante. La azul usa un punto a a cada lado; para esta parábola coincide con la tangente verde exacta.

En a : altura $f(a)$

— paso h →

En $a + h$: altura $f(a + h)$

5.1 La trampa de hacer h demasiado pequeño

Parece que un paso diminuto siempre debería ser mejor. No es así:

- Si h es grande, vemos demasiada montaña y perdemos detalle local.
- Si h es minúsculo, restamos números casi iguales y la computadora pierde dígitos útiles.

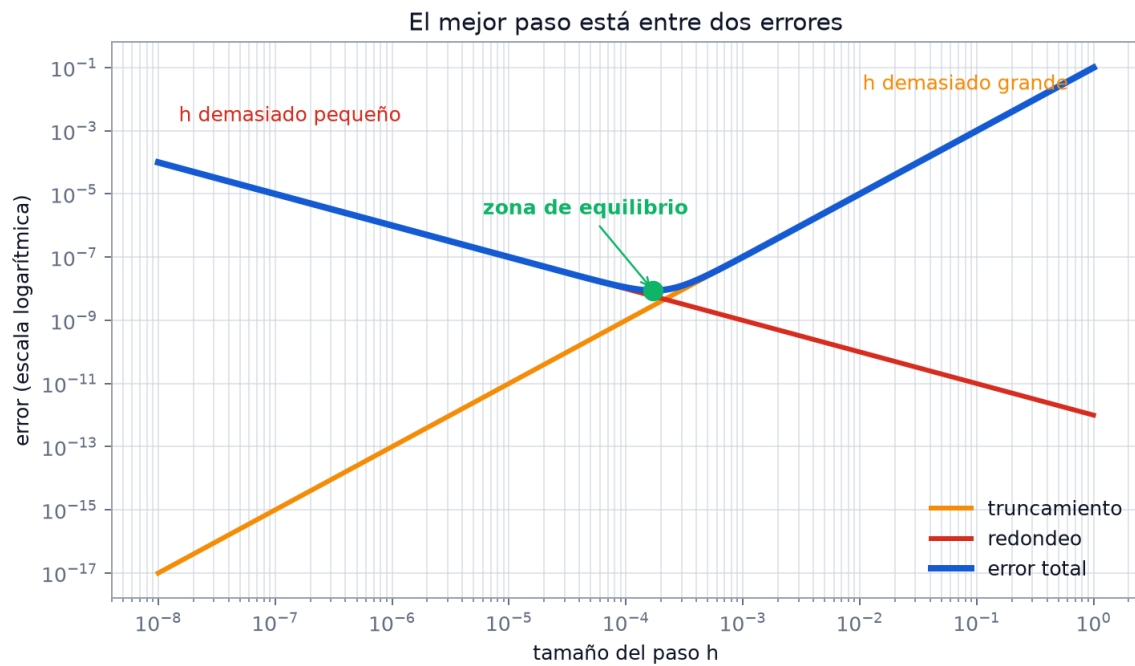


Figura 4: **El error tiene dos enemigos.** Al reducir h baja el truncamiento naranja, pero crece el redondeo rojo. El punto verde marca la zona de equilibrio. Los valores son ilustrativos; la forma de “U” es la lección.

h grande	h razonable	h microscópico
Error de truncamiento	Buen equilibrio	Error de redondeo
		

5.2 Dos variables

Para x usamos un paso h_x . Para y usamos h_y . Una derivada mixta se construye evaluando una pequeña cuadrícula alrededor del centro y combinando sus alturas con pesos positivos y negativos.

✓ Comprobación rápida

Diferencias finitas = estimar derivadas mirando valores cercanos. No se usa una derivada simbólica escondida.

6 Pascal: una máquina para repartir

El triángulo de Pascal empieza con un 1. Cada nuevo número interior es la suma de los dos que tiene encima:

$$\begin{array}{ccccccc}
 & & & & 1 & & & \\
 & & & 1 & & 1 & & \\
 & & 1 & & 2 & & 1 & \\
 & 1 & & 3 & & 3 & & 1 \\
 1 & & 4 & & 6 & & 4 & & 1 \\
 1 & & 5 & & 10 & & 10 & & 5 & & 1
 \end{array}$$

Pascal: cada círculo interior suma sus dos padres

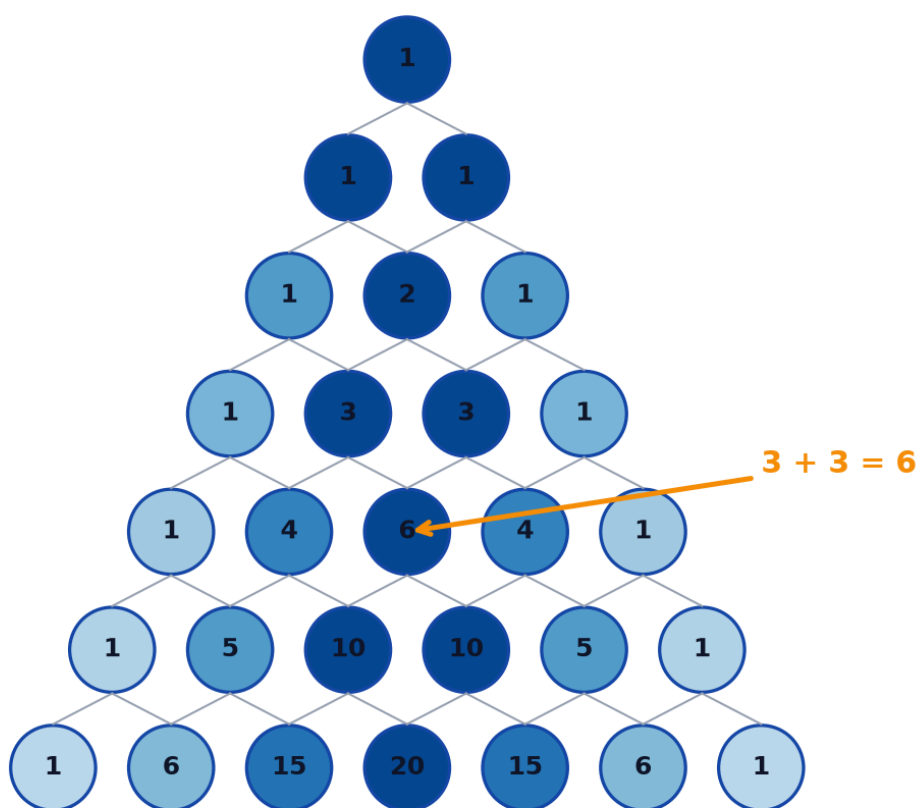


Figura 5: **Pascal construido, no memorizado.** Las líneas muestran los dos padres de cada número interior. Por ejemplo, los dos 3 de arriba producen el 6.

6.1 ¿Qué tiene que ver con x e y ?

En el orden n , debemos repartir n derivadas entre x e y .

Para $n = 3$ existen cuatro repartos:

xxx	xyx	xyy	yyy
f_{xxx}	f_{xyx}	f_{xyy}	f_{yyy}
1	3	3	1

¡La fila es precisamente 1, 3, 3, 1!

6.2 El binomial

El número en la posición i de la fila n se escribe:

$$\binom{n}{i}$$

El programa **construye** todas las filas sumando vecinos. Después lee esos números para ensamblar Taylor. No necesita una tabla guardada de antemano.



Dos trabajos de Pascal

En beginner, Pascal crea los pesos de las diferencias progresivas y también los coeficientes de Taylor. En advanced y hero, Fornberg crea los pesos numéricos, pero Pascal sigue ensamblando todos los términos de Taylor.

7 La fórmula de Taylor, pieza por pieza

Primero la mostramos completa. No intentes memorizarla:

$$T_{N(x,y)} = \sum_{n=0}^N \sum_{i=0}^n \frac{\binom{n}{i}}{n!} \frac{\partial^n f}{\partial x^i \partial y^{n-i}}(a,b) (x-a)^i (y-b)^{n-i}$$

Ahora la desarmamos.

7.1 $T_{N(x,y)}$

Es el nombre de nuestra copia sencilla. La letra T recuerda a Taylor y N indica el mayor orden incluido.

7.2 Las dos sumas

- La suma exterior recorre los órdenes $0, 1, 2, \dots, N$.
- La suma interior recorre todas las formas de repartir cada orden entre x e y .

7.3 $\binom{n}{i}$

Es el número tomado de Pascal. Dice cuántas veces aparece ese reparto.

7.4 $n!$

El factorial equilibra el tamaño de los términos:

$$0! = 1, \quad 1! = 1, \quad 2! = 2, \quad 3! = 6, \quad 4! = 24$$

7.5 La derivada parcial

Cuenta la forma local de la función en el centro. En este proyecto se obtiene mediante diferencias finitas.

7.6 $(x-a)^i (y-b)^{n-i}$

Cuenta cuánto nos hemos alejado del centro. En el centro, las potencias de orden positivo desaparecen, por eso

$$T_{N(a,b)} = f(a,b).$$

✓ Comprobación rápida

Cada término = número de Pascal × información local × distancia al centro, con un factorial para mantener la escala correcta.

8 Un ejemplo completo que cabe en una hoja

Tomemos:

$$f(x, y) = x^2 + 2xy + 3y^2, \quad (a, b) = (0, 0), \quad N = 2$$

8.1 Paso 1: valor en el centro

$$f(0, 0) = 0$$

8.2 Paso 2: primeras derivadas

$$f_x = 2x + 2y, \quad f_y = 2x + 6y$$

En el centro ambas valen cero.

8.3 Paso 3: segundas derivadas

$$f_{xx} = 2, \quad f_{xy} = 2, \quad f_{yy} = 6$$

8.4 Paso 4: fila 2 de Pascal

$$1 \quad 2 \quad 1$$

8.5 Paso 5: ensamblar

$$T_2(x, y) = \frac{1}{2!}2x^2 + \frac{2}{2!}2xy + \frac{1}{2!}6y^2$$

Simplificando:

$$T_2(x, y) = x^2 + 2xy + 3y^2$$

¡Es exactamente la función original! Esto ocurre porque la función ya era un polinomio de grado 2 y elegimos $N = 2$.



¿Y si la función es seno o exponencial?

Entonces Taylor no suele ser idéntico en todas partes. Se parece mucho cerca del centro y mejora localmente al incluir más órdenes, siempre que las derivadas se calculen con suficiente precisión.

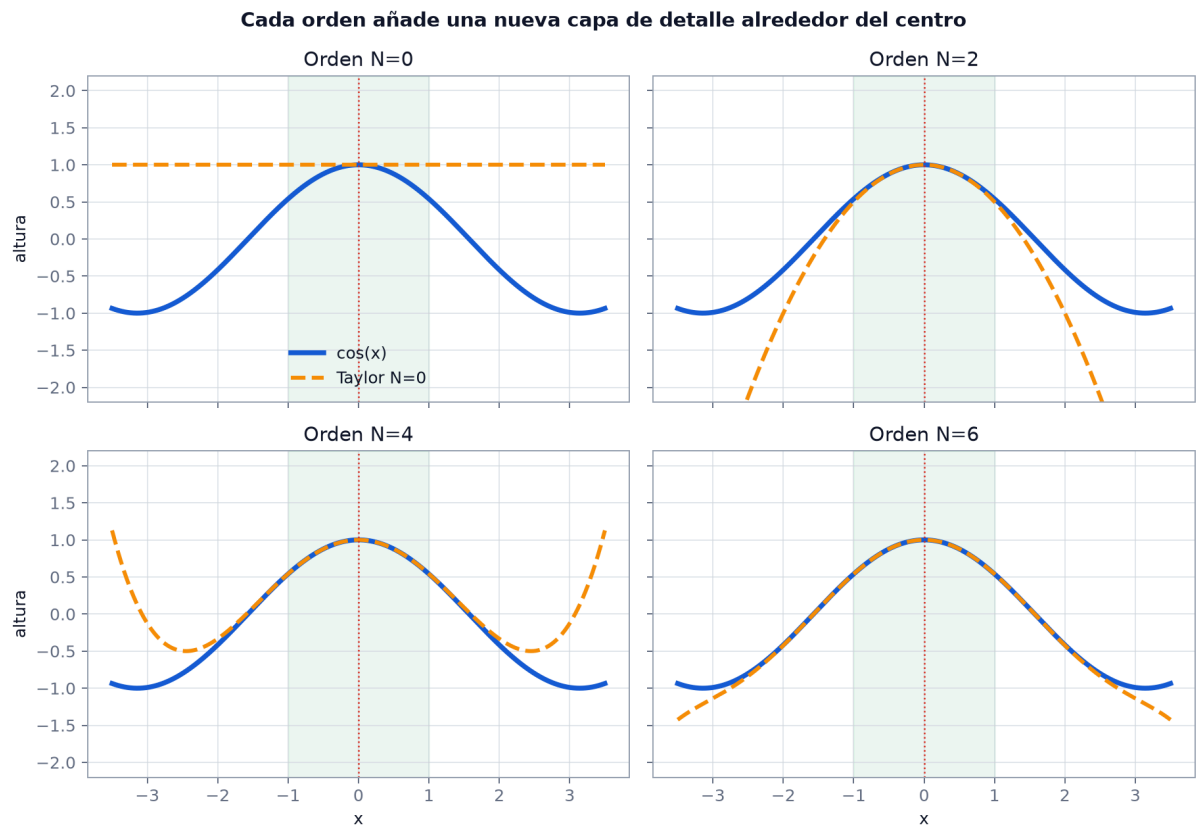


Figura 6: **Agregar órdenes añade detalle alrededor del centro.** La franja verde es la zona local. De $N = 0$ a $N = 6$, la curva naranja imita cada vez más a la azul antes de separarse lejos de cero.

9 Los tres niveles del programa

Los niveles no son tres nombres para lo mismo. Cada uno cambia la estrategia numérica y tiene un propósito claro.

9.1 Nivel 1 — beginner

Objetivo: ver toda la mecánica

Usa diferencias progresivas. Sus pesos vienen directamente de Pascal:

$$D_x^i D_y^j f(a, b) \approx \frac{1}{h_x^i h_y^j} \sum_{p=0}^i \sum_{q=0}^j (-1)^{i-p+j-q} \binom{i}{p} \binom{j}{q} f(a + ph_x, b + qh_y)$$

Es fácil de seguir con lápiz. Su error es aproximadamente $O(h)$, así que no es la mejor opción para resultados exigentes.

Elige beginner cuando: estás aprendiendo, explicando en clase o usando un N pequeño.

9.2 Nivel 2 — advanced

Objetivo: rapidez y buena precisión

1. Coloca nodos simétricos a ambos lados del centro.
2. Genera pesos de cualquier orden con la recurrencia de Fornberg.
3. Combina pesos de x e y como una cuadrícula.
4. Evalúa la función una sola vez por punto y guarda el resultado.
5. Repite con h , $\frac{h}{2}$ y $\frac{h}{4}$.
6. Usa extrapolación de Richardson para reducir el error.
7. Reporta una incertidumbre estimada.

Todo se calcula con números normales de 64 bits (float64).

Elige advanced cuando: quieres el modo recomendado para órdenes moderados.

9.3 Nivel 3 — hero

Objetivo: sobrevivir a órdenes altos

Repite exactamente la estrategia de advanced, pero usa números con 80, 120 o más dígitos internos mediante `mpmath`.

Esto combate la cancelación: en una derivada de orden alto aparecen sumas de números grandes con signos contrarios cuyo resultado puede ser pequeño.

Elige hero cuando: N es alto o advanced muestra derivadas inestables.

⚠ Hero no hace magia

Más dígitos ayudan contra el redondeo, pero no arreglan una función no suave, un punto fuera del dominio o una expansión demasiado lejos del centro.

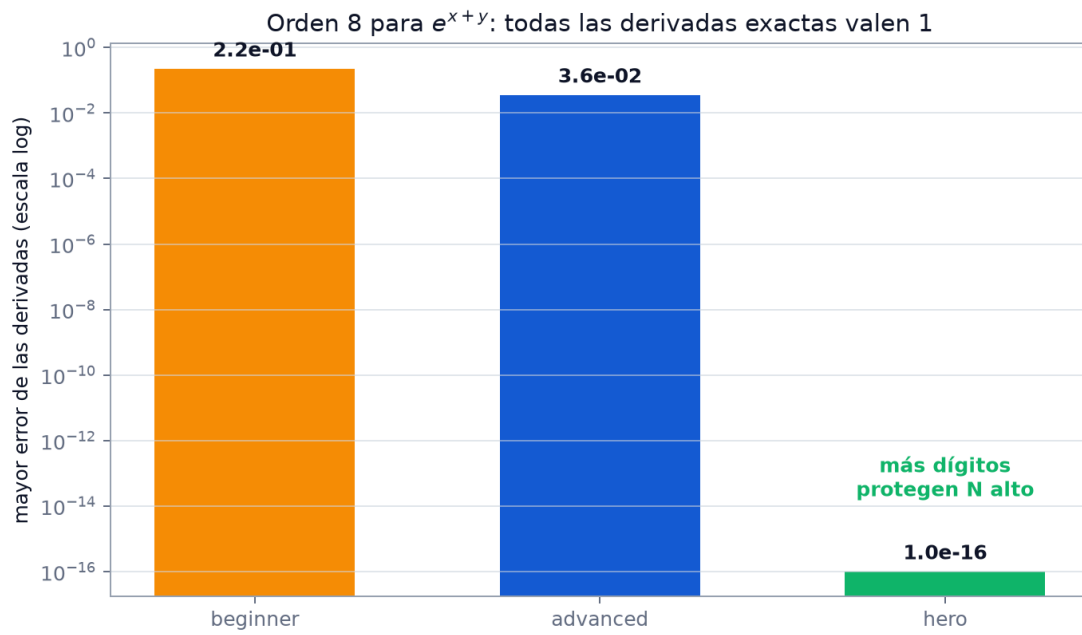


Figura 7: **Una prueba con respuesta conocida.** Para e^{x+y} todas las derivadas en el origen valen 1. En orden 8, la precisión arbitraria de hero conserva muchos más dígitos. El eje vertical es logarítmico.

10 Fornberg sin misterio

Queremos números w_k que cumplan:

$$f^m(0) \approx \sum_k w_k f(z_k)$$

Los z_k son posiciones de muestreo y los w_k son pesos. Por ejemplo, para la primera derivada con nodos $-1, 0, 1$:

$$(w_{-1}, w_0, w_1) = \left(-\frac{1}{2}, 0, \frac{1}{2}\right)$$

Al incorporar el paso h obtenemos la conocida diferencia centrada:

$$f'(a) \approx \frac{f(a+h) - f(a-h)}{2h}$$

10.1 ¿Por qué no resolver un sistema gigante?

Podríamos calcular los pesos resolviendo ecuaciones con potencias de los nodos, pero esos sistemas se vuelven numéricamente incómodos. La recurrencia de Fornberg construye los pesos progresivamente y admite:

- cualquier orden de derivada;
- cualquier cantidad suficiente de nodos;
- nodos uniformes o irregulares;
- aritmética normal o precisión arbitraria.

10.2 Derivadas mixtas como producto tensorial

Si w_x contiene pesos para x y w_y contiene pesos para y , entonces:

$$D_x^i D_y^j f \approx \sum_p \sum_q w_{x_p} w_{y_q} f(x_p, y_q)$$

Es como poner una regla horizontal encima de una regla vertical. El programa guarda las alturas de toda la cuadrícula y las reutiliza para las distintas derivadas.

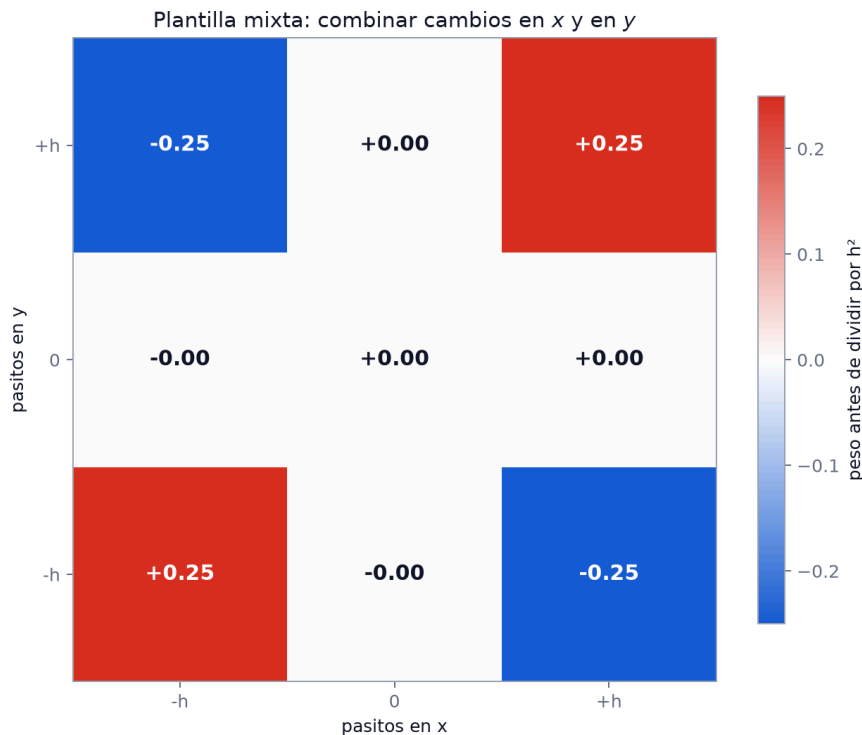


Figura 8: Una plantilla para f_{xy} . Rojo significa sumar, azul restar y blanco peso cero. Las cuatro esquinas comparan cómo cambia la función en ambas direcciones a la vez.

11 Richardson: comparar tres tamaños de paso

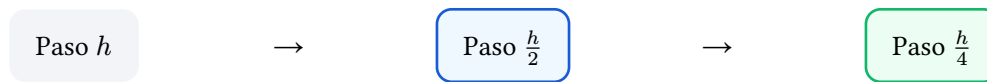
Supongamos que un cálculo tiene un error principal proporcional a h^p :

$$A(h) = A_{\text{real}} + Ch^p + \text{errores menores}$$

Calculamos $A(h)$ y $A(\frac{h}{2})$. Como sabemos cómo cambia h^p , podemos cancelar gran parte de ese error:

$$A_{\text{mejor}} = A\left(\frac{h}{2}\right) + \frac{A(\frac{h}{2}) - A(h)}{2^p - 1}$$

El programa hace esto dos veces:

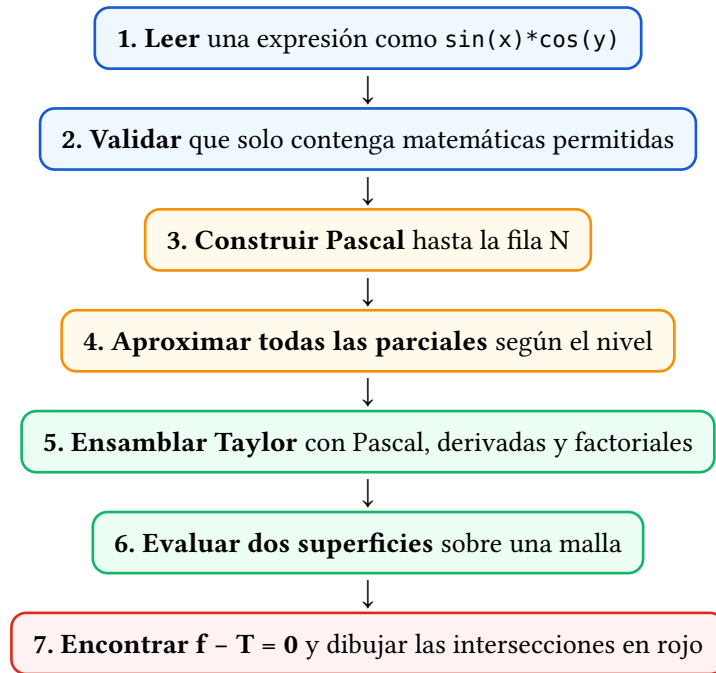


Si al reducir el paso la estimación empeora, el programa conserva la etapa anterior. Así evita perseguir pasos pequeños cuando el redondeo ya manda.

11.1 Qué significa “incertidumbre” en la tabla

Es una estimación construida comparando refinamientos. Sirve como alarma y para comparar términos. **No es una demostración rigurosa** del error total.

12 Qué hace el código, de principio a fin



12.1 Archivos principales

Archivo	Responsabilidad
src/taylor_fd/core.py	Pascal, Fornberg, los tres niveles y el modelo.
src/taylor_fd/expressions.py	Intérprete restringido de funciones.
src/taylor_fd/plotting.py	Mallas, métricas, superficies e intersecciones.
src/taylor_fd/cli.py	Argumentos y mensajes en español.
tests/	Pruebas matemáticas, seguridad e integración.

13 Cómo se encuentra una intersección

Las superficies se intersectan cuando tienen la misma altura:

$$f(x, y) = T_{N(x, y)}$$

Movemos todo a un lado:

$$d(x, y) = f(x, y) - T_{N(x, y)}$$

Entonces buscamos lugares donde $d(x, y) = 0$.

13.1 Del mapa 2D a la curva 3D

1. Evaluamos d en una cuadrícula de puntos (x, y) .
2. Un algoritmo de contornos conecta los lugares donde el signo cambia.
3. Así obtenemos curvas en el plano (x, y) .
4. Calculamos $z = f(x, y)$ sobre cada curva.
5. Dibujamos esas coordenadas (x, y, z) en rojo.



Detalle importante

La línea roja no se dibuja pegada al suelo como una sombra. Se eleva hasta la altura común de las dos superficies; por eso representa una intersección 3D.

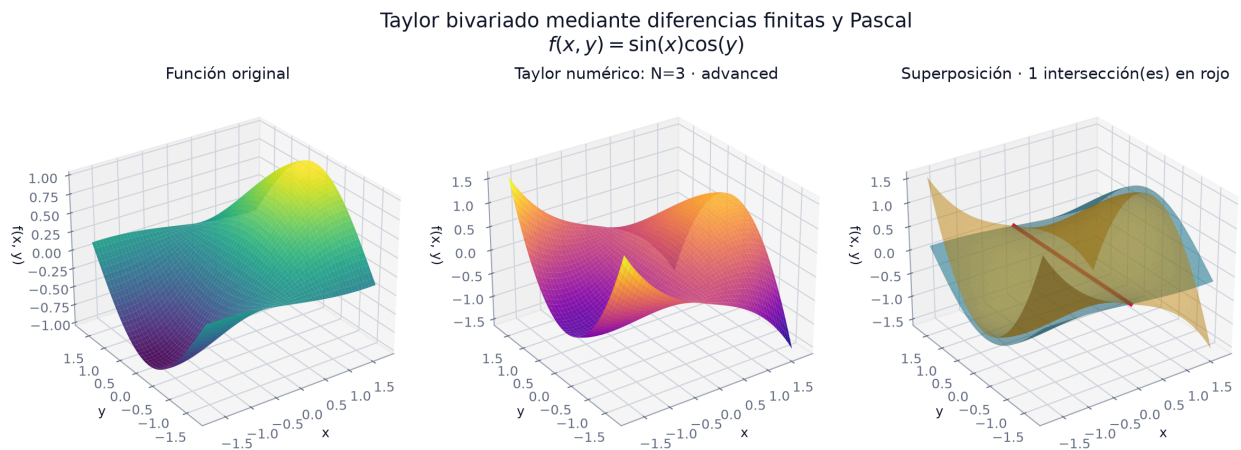


Figura 9: **Salida real del programa.** Izquierda: original. Centro: Taylor. Derecha: ambas superficies transparentes; la línea roja vive a la altura común y marca dónde se intersectan.

13.2 Las métricas

En la misma malla se calcula el error $e = f - T$.

$$\text{RMSE} = \sqrt{\frac{1}{M} \sum_{k=1}^M e_k^2}$$

- **RMSE:** tamaño típico del error, dando más peso a errores grandes.
- **Error máximo:** el peor punto observado en la malla.

Estas métricas dependen del rango y de la resolución elegidos.

14 Qué significa “cualquier función”

La API de Python acepta cualquier objeto invocable $f(x, y)$ que devuelva un número y funcione en los puntos de la plantilla.

La terminal acepta expresiones con:

Variables: x, y

Constantes: π, e

Operadores: $+$ $-$ $*$ $/$ $**$

Funciones: $\sin$ $\cos$ $\tan$ asin acos atan $\sinh$ $\cosh$ $\tanh$ $\exp$ $\log$ $\sqrt{}$ abs

14.1 Seguridad del intérprete

El texto no pasa directamente a `eval`. Se convierte en un árbol de sintaxis y solo se aceptan nodos matemáticos de una lista explícita. Se rechazan atributos, imports, archivos, llamadas desconocidas y variables distintas de x e y .

14.2 Requisitos matemáticos reales

Para pedir orden N , la función debe tener las derivadas correspondientes en la vecindad usada. Ejemplos problemáticos:

- $\operatorname{abs}(x)$ en $x = 0$: tiene una esquina.
- $\log(x)$ alrededor de un centro cuya plantilla toca $x \leq 0$.
- $1/x$ alrededor de $x = 0$.
- una función con saltos.



El dominio también incluye los pasitos

No basta con que la función exista exactamente en el centro. Debe existir en todos los puntos cercanos que use la plantilla finita.

15 Uso del programa con uv

15.1 Instalación reproducible

```
uv sync
```

`uv.lock` fija las versiones resueltas para que distintas computadoras instalen el mismo conjunto de dependencias.

15.2 Ejemplo recomendado

```
uv run taylor \
  --funcion "sin(x)*cos(y)" \
  --centro 0 0 \
  -N 6 \
  --nivel advanced
```

15.3 Ejemplo hero

```
uv run taylor \
  --funcion "sin(x*y)*exp(x-y)" \
  --centro 0.1 0.2 \
  -N 10 \
  --nivel hero \
  --digitos 120 \
  --precision-plantilla 8
```

15.4 Servidor sin pantalla

```
uv run taylor --no-mostrar --salida mi_grafica.png
```

15.5 Argumentos más importantes

Opción	Significado
<code>--funcion</code>	Expresión de $f(x, y)$.
<code>--orden</code> o <code>-N</code>	Mayor orden total de Taylor.
<code>--nivel</code>	beginner, advanced o hero.
<code>--centro A B</code>	Punto de expansión.
<code>--paso h</code>	Paso común; también admite h_x h_y .
<code>--digitos</code>	Precisión decimal interna de hero.
<code>--rango-x</code> , <code>--rango-y</code>	Ventana mostrada.
<code>--puntos</code>	Resolución por eje de la gráfica.

16 Cómo elegir N , h y el rango

No existe un único valor perfecto para todas las funciones.

16.1 Elección práctica de N

- Empieza con $N = 3$ o $N = 4$.
- Aumenta gradualmente y observa las métricas cerca del centro.
- Si las derivadas altas se vuelven absurdas en advanced, usa hero.
- Más términos no garantizan una mejor aproximación en un rango enorme.

16.2 Elección práctica de h

- El valor predeterminado avanzado es 0.1 y se refina internamente.
- Si la función cambia en escalas minúsculas, prueba un paso menor.
- Si la función cambia muy lentamente o el centro es enorme, prueba un paso proporcional a esa escala.
- Mira la columna de incertidumbre.

16.3 Elección práctica del rango

Empieza viendo una caja pequeña alrededor del centro. Después amplíala. Esto separa dos preguntas:

1. ¿Las derivadas están bien calculadas?
2. ¿Hasta dónde es buena esta aproximación local?

17 Tests: cómo sabemos que funciona

La suite no compara solamente capturas de pantalla. Comprueba propiedades matemáticas con respuestas conocidas.

17.1 Pruebas unitarias

- Pascal produce la fila 1 5 10 10 5 1 para $N = 5$.
- Fornberg produce $(-\frac{1}{2}, 0, \frac{1}{2})$ para la primera derivada centrada.
- Fornberg produce $(1, -2, 1)$ para la segunda derivada centrada.
- Los tres niveles reconstruyen un polinomio conocido dentro de su precisión.
- Las derivadas mixtas de $e^x \sin(y)$ coinciden con sus valores exactos.
- hero conserva derivadas mixtas de orden alto de e^{x+y} .

17.2 Pruebas de seguridad

El parser rechaza intentos de importar módulos, abrir archivos, acceder a atributos o usar símbolos desconocidos.

17.3 Pruebas de integración

- La CLI completa construye un modelo y guarda un PNG válido.
- La visualización devuelve métricas finitas.
- El fuente Typst se compila y el PDF generado comienza con una cabecera PDF válida.
- La CI ejecuta Ruff, formato, Pytest y compilación de esta guía.

17.4 Comandos de calidad

```
uv run ruff check .
uv run ruff format --check .
uv run pytest
typst compile docs/guia_eli5.typ docs/guia_eli5.pdf
```

✓ Comprobación rápida

Una prueba no demuestra que toda función imaginable funcionará, pero cada propiedad conocida protege una pieza crítica del algoritmo.

18 Cuaderno de ejercicios: de cero a hero

Esta ruta está diseñada para hacerse en orden. No mires las soluciones hasta haber escrito o dicho en voz alta un intento. Equivocarse aquí es parte del método.



Cómo estudiar con estos ejercicios

1. Responde sin computadora.
2. Marca lo que no entiendas.
3. Usa la pista si pasan cinco minutos.
4. Comprueba la solución.
5. Explica la respuesta con tus propias palabras. Si puedes explicarla, ya no la estás memorizando.

18.1 Nivel 0 — Intuición, sin fórmulas

18.1.1 Ejercicio 0.1 · El mapa y la altura

Una función recibe (x, y) y devuelve $f(x, y)$. En la analogía de la montaña, ¿qué representa cada una de esas tres cosas?

Pista: dos indican dónde estás y una indica qué tan arriba estás.

18.1.2 Ejercicio 0.2 · La lupa

Taylor copia muy bien una región pequeña alrededor de (a, b) . Si quieres estudiar la superficie cerca de $(10, -3)$, ¿qué centro elegirías? ¿Por qué no usar automáticamente $(0, 0)$?

18.1.3 Ejercicio 0.3 · Dos caminos

En una superficie puedes caminar hacia x o hacia y . Si la altura aumenta al caminar hacia x y no cambia al caminar hacia y , ¿qué derivada primera parece positiva y cuál parece cero?

18.1.4 Ejercicio 0.4 · Copia local

Una aproximación coincide muy bien con la función en el centro, pero se separa en los bordes de una gráfica enorme. ¿Es suficiente para declarar que el código está mal? Propón una comprobación mejor.

18.1.5 Ejercicio 0.5 · Detecta la función problemática

¿Cuál es más amigable para un Taylor alrededor de $(0, 0)$ y por qué?

1. $f(x, y) = x^2 + y^2$
2. $f(x, y) = |x| + y^2$

Pista: imagina la forma de $\text{abs}(x)$ justo en cero.

18.2 Nivel 1 — Pascal y cambios pequeños

18.2.1 Ejercicio 1.1 · Construye Pascal

Sin consultar la guía, escribe las filas 0 a 5 del triángulo de Pascal. Recuerda que los extremos son 1 y cada interior suma dos vecinos de arriba.

18.2.2 Ejercicio 1.2 · Repartir cuatro derivadas

Escribe todas las derivadas parciales de orden total 4, desde “todo en x ” hasta “todo en y ”. Después coloca debajo los números de la fila 4 de Pascal.

18.2.3 Ejercicio 1.3 · Una pendiente con datos

Sabemos que $f(2) = 7$ y $f(2.1) = 7.42$. Usa una diferencia progresiva con $h = 0.1$ para estimar $f'(2)$.

18.2.4 Ejercicio 1.4 · Diferencia centrada

Sabemos que $f(1.9) = 3.61$ y $f(2.1) = 4.41$. Estima $f'(2)$ con:

$$\frac{f(2+h) - f(2-h)}{2h}$$

¿Reconoces una función sencilla compatible con esos valores?

18.2.5 Ejercicio 1.5 · El paso imposible

Explica por qué estas dos frases son falsas:

- “Un h gigantesco siempre es mejor porque evita redondeo”.
- “Un h tan pequeño como sea posible siempre es mejor”.

18.3 Nivel 2 — Construir Taylor con las manos

18.3.1 Ejercicio 2.1 · Contar términos

¿Cuántos términos tiene un Taylor bivariado total hasta $N = 3$? Usa la fórmula $(N + 1)\frac{N+2}{2}$ y comprueba contando por órdenes.

18.3.2 Ejercicio 2.2 · Una función lineal

Para $f(x, y) = 3 + 2x - y$, centro $(0, 0)$ y $N = 1$:

1. calcula $f(0, 0)$, f_x y f_y ;
2. construye $T_1(x, y)$;
3. explica por qué coincide en todas partes.

18.3.3 Ejercicio 2.3 · Un tazón desplazado

Para $f(x, y) = x^2 + y^2$, centro $(1, 1)$ y $N = 2$, calcula:

$$f(1, 1), f_{x(1,1)}, f_{y(1,1)}, f_{xx}, f_{xy}, f_{yy}$$

Luego escribe el Taylor usando $\Delta x = x - 1$ y $\Delta y = y - 1$.

18.3.4 Ejercicio 2.4 · Usa Pascal de verdad

En el término de orden 3, ¿qué multiplicadores de Pascal acompañan a f_{xxx} , f_{xxy} , f_{xyy} y f_{yyy} ? ¿Por qué todos se dividen entre 3!?

18.3.5 Ejercicio 2.5 · Evalúa una aproximación

Supón que alrededor de $(0, 0)$ obtuviste:

$$T_2(x, y) = 1 + 2x - 3y + \frac{1}{2}x^2 + 4xy - y^2$$

Calcula $T_2(0.1, -0.2)$ paso a paso.

18.4 Nivel 3 — Usar y comprobar el programa

18.4.1 Ejercicio 3.1 · Primera ejecución

Ejecuta:

```
uv run taylor \
--funcion "x**2 + 2*x*y + 3*y**2" \
-N 2 \
--nivel advanced \
--rango-x -1 1 \
--rango-y -1 1 \
--no-mostrar \
--salida ejercicio_3_1.png
```

Predice el RMSE antes de mirar la terminal. Después explica cualquier pequeña diferencia respecto de cero.

18.4.2 Ejercicio 3.2 · Tres niveles

Ejecuta la misma función $\exp(x-y)$ con $N = 4$ y los niveles beginner, advanced y hero. Conserva el mismo centro y rango. Anota:

- RMSE;
- error máximo;
- incertidumbre de dos derivadas;
- tiempo aproximado.

¿Qué nivel elegirías para una demostración? ¿Y para un cálculo?

18.4.3 Ejercicio 3.3 · Cambiar el centro

Aproxima $\sin(x)\cos(y)$ primero alrededor de $(0, 0)$ y luego alrededor de $(1, 1)$. Mantén el rango cerca de $(1, 1)$. ¿Cuál modelo funciona mejor allí? Explica sin decir solamente “porque el número fue menor”.

18.4.4 Ejercicio 3.4 · Local contra global

Con función $\exp(-(x^2+y^2))$, centro $(0, 0)$ y $N = 6$, compara los rangos:

- $[-0.5, 0.5]$ en ambos ejes;
- $[-2, 2]$ en ambos ejes.

¿Cambiaron las derivadas? ¿Cambió el error sobre la malla? ¿Por qué?

18.4.5 Ejercicio 3.5 · Seguridad

Prueba la expresión siguiente y lee el error:

```
uv run taylor --funcion "__import__('os').system('echo NO')"
```

Explica por qué rechazarla es una característica, no una limitación matemática.

18.5 Nivel 4 — Diagnóstico hero

18.5.1 Ejercicio 4.1 · Derivadas que conocemos

Para $f(x, y) = e^{x+y}$, toda derivada parcial en $(0, 0)$ vale 1. Ejecuta con $N = 10$ en advanced y hero. Busca en la tabla las derivadas $(5, 5)$, $(9, 1)$ y $(0, 10)$. ¿Cuál nivel conserva mejor el valor esperado?

18.5.2 Ejercicio 4.2 · Más pequeño no siempre gana

En advanced, prueba manualmente `--paso 0.5`, `--paso 0.1`, `--paso 0.001` y `--paso 0.000001` para una derivada de orden alto. Describe la forma general del error. ¿Dónde domina truncamiento y dónde redondeo?

18.5.3 Ejercicio 4.3 · Suavidad

Compara $\sqrt{x^2+y^2}$ alrededor de $(0, 0)$ y alrededor de $(1, 1)$. ¿Por qué el mismo código puede comportarse de manera muy distinta?

18.5.4 Ejercicio 4.4 · Diseña un experimento justo

Quieres comparar advanced y hero. Escribe un protocolo que mantenga fijos función, centro, N , paso, precisión de plantilla, rango y resolución. Indica qué métricas registrarías y qué conclusión **no** podrías sacar de una sola función.

18.5.5 Ejercicio 4.5 · Explica todo en 60 segundos

Sin mirar la guía, explica a otra persona estas seis palabras y conéctalas en una historia: **centro**, **derivada**, **diferencia finita**, **Pascal**, **Taylor**, **intersección**. Si falta una conexión, vuelve al capítulo correspondiente.

19 Soluciones razonadas de los ejercicios

No basta con comparar el resultado final: revisa el camino.

19.1 Soluciones del nivel 0

19.1.1 Solución 0.1

x es la posición este-oeste, y la posición norte-sur y $f(x, y)$ la altura. Las dos entradas localizan un punto del mapa; la salida levanta ese punto a 3D.

19.1.2 Solución 0.2

Elegiríamos $(a, b) = (10, -3)$. Taylor usa información local del centro. El origen podría estar lejos de la zona relevante, así que sus potencias describirían bien otra vecindad.

19.1.3 Solución 0.3

f_x parece positiva y f_y parece cero. “Mantener la otra variable quieta” es esencial al interpretar una derivada parcial.

19.1.4 Solución 0.4

No. Primero hay que reducir el rango alrededor del centro y comprobar derivadas contra una función conocida. Taylor es local; el borde lejano prueba alcance, no solamente corrección numérica.

19.1.5 Solución 0.5

$x^2 + y^2$ es suave en el origen. $\text{abs}(x)$ tiene una esquina allí: sus pendientes izquierda y derecha no coinciden, así que no existe la derivada clásica en x .

19.2 Soluciones del nivel 1

19.2.1 Solución 1.1

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

19.2.2 Solución 1.2

$f_{xxxx}, f_{xxxy}, f_{xxyy}, f_{xyyy}$ y f_{yyyy} . Debajo van 1, 4, 6, 4, 1.

19.2.3 Solución 1.3

$$f'(2) \approx \frac{7.42 - 7}{0.1} = 4.2$$

Es una estimación, no necesariamente el valor exacto.

19.2.4 Solución 1.4

$$f'(2) \approx \frac{4.41 - 3.61}{0.2} = 4$$

Los datos son compatibles con $f(x) = x^2$, cuya derivada exacta en 2 es 4.

19.2.5 Solución 1.5

Con h grande, la secante observa una región demasiado amplia: domina el error de truncamiento. Con h microscópico se restan valores casi iguales y se amplifica el redondeo al dividir entre h . Buscamos equilibrio.

19.3 Soluciones del nivel 2

19.3.1 Solución 2.1

$(3 + 1)^{\frac{3+2}{2}} = 10$. Por órdenes contamos $1 + 2 + 3 + 4 = 10$.

19.3.2 Solución 2.2

$f(0, 0) = 3$, $f_x = 2$, $f_y = -1$. Por tanto $T_1 = 3 + 2x - y$. Coincide globalmente porque la función original ya era un polinomio de grado 1.

19.3.3 Solución 2.3

$f(1, 1) = 2$, $f_x = 2x$ da 2, $f_y = 2y$ da 2, $f_{xx} = 2$, $f_{xy} = 0$ y $f_{yy} = 2$. Entonces:

$$T_2 = 2 + 2\Delta x + 2\Delta y + \Delta x^2 + \Delta y^2$$

Al sustituir $\Delta x = x - 1$ y $\Delta y = y - 1$ se recupera $x^2 + y^2$.

19.3.4 Solución 2.4

Los multiplicadores son 1, 3, 3, 1. Se dividen entre $3 \neq 6$ porque todos pertenecen al orden total 3; el factorial aparece en la expansión exponencial del operador de desplazamiento y normaliza ese orden.

19.3.5 Solución 2.5

Con $x = 0.1$, $y = -0.2$:

$$1 + 0.2 + 0.6 + 0.005 - 0.08 - 0.04 = 1.685$$

El término $4xy$ es negativo porque x e y tienen signos opuestos.

19.4 Soluciones del nivel 3

19.4.1 Solución 3.1

Debe ser cero salvo ruido de punto flotante, porque un Taylor de orden 2 reconstruye exactamente ese polinomio cuadrático. Un resultado alrededor de 10^{-14} o menor es numéricamente cero en `float64`.

19.4.2 Solución 3.2

Los números exactos dependen de plataforma y rango. La conclusión esperada es: `beginner` muestra con claridad el procedimiento pero tiene mayor error; `advanced` ofrece el mejor equilibrio; `hero` protege órdenes altos a cambio de tiempo. La incertidumbre de `beginner` aparece como “—” porque no refina pasos.

19.4.3 Solución 3.3

El modelo centrado en $(1, 1)$ debería funcionar mejor en una vecindad de $(1, 1)$ porque sus derivadas y distancias se organizan localmente allí. El centrado en cero intenta extender una descripción creada en otra zona.

19.4.4 Solución 3.4

Las derivadas del modelo no cambian: función, centro y parámetros numéricos son los mismos. Sí cambia el error medido porque la segunda malla incluye puntos más lejanos, donde los términos omitidos de Taylor pesan más.

19.4.5 Solución 3.5

El parser debe rechazarla antes de ejecutar nada. La CLI promete matemáticas, no acceso al sistema operativo. Limitar la sintaxis mantiene esa frontera.

19.5 Soluciones del nivel 4

19.5.1 Solución 4.1

El valor teórico de las tres derivadas es 1. En orden 10, `float64` suele perder exactitud por cancelación, mientras `hero` conserva muchos más dígitos gracias a la precisión arbitraria. El resultado concreto depende de paso y plantilla.

19.5.2 Solución 4.2

La curva de error suele bajar y luego subir: con pasos grandes domina truncamiento; en una zona intermedia hay equilibrio; con pasos extremadamente pequeños domina redondeo/cancelación. Richardson automatiza parte de esta comparación usando tres escalas.

19.5.3 Solución 4.3

$\sqrt{x^2 + y^2}$ es la distancia al origen y forma un cono con punta en $(0, 0)$. Allí no es diferenciable de la manera requerida. Alrededor de $(1, 1)$ la superficie es suave y las diferencias finitas tienen una vecindad regular.

19.5.4 Solución 4.4

El protocolo debe cambiar solamente el nivel, registrar derivadas conocidas, incertidumbre, RMSE, error máximo y tiempo, y repetir para evitar ruido de medición. Una función sola no demuestra superioridad universal: escalas, suavidad, coste y orden cambian el problema.

19.5.5 Solución 4.5

Una respuesta posible: “Elegimos un **centro**. Medimos allí los cambios o **derivadas** con pequeños muestreos llamados **diferencias finitas**. El triángulo de **Pascal** reparte cada orden entre x e y . Con todo armamos el polinomio de **Taylor**. Finalmente buscamos dónde original y copia tienen la misma altura; esas son sus **intersecciones**”.

20 Preguntas frecuentes

20.1 ¿SymPy calcula las derivadas?

No. SymPy ayuda a representar la expresión y crear evaluadores para NumPy o mpmath. Las derivadas del modelo se calculan mediante diferencias finitas.

20.2 ¿Pascal calcula las derivadas en todos los niveles?

En beginner, sí participa directamente en sus pesos. En advanced y hero, los pesos se generan con Fornberg. En **todos** los niveles Pascal entrega los coeficientes binomiales que ensamblan Taylor.

20.3 ¿Por qué mi error crece lejos del centro?

Porque Taylor es local. Prueba un rango menor o mueve el centro hacia la región que te interesa.

20.4 ¿Por qué aumentar N empeoró el resultado?

Las derivadas altas son sensibles al redondeo y al paso. Prueba hero, más dígitos, otra precisión de plantilla o un rango menor. También verifica que la función sea suave.

20.5 ¿Una línea roja significa infinitos puntos de igualdad?

Sí: cada curva aproximada reúne muchos puntos donde las dos alturas coinciden. La precisión geométrica está limitada por la resolución de la malla.

20.6 ¿Puedo usar una función escrita directamente en Python?

Sí:

```
from taylor_fd import build_taylor_model

def mi_funcion(x, y):
    return x**2 * y + y**3

modelo = build_taylor_model(
    mi_funcion,
    center=(0.0, 0.0),
    order=3,
    level="advanced",
)

print(modelo.evaluate(0.2, 0.4))
```

21 Glosario de bolsillo

Callable: Algo que puede llamarse como $f(x, y)$.

Centro: Punto (a, b) alrededor del cual copiamos la función.

Coficiente: Número que multiplica una pieza del polinomio.

Derivada: Medida local de cambio.

Derivada mixta: Cambio que involucra tanto x como y .

Diferencia finita: Estimación de una derivada usando valores cercanos.

Error de redondeo: Pérdida causada por representar números con dígitos finitos.

Error de truncamiento: Error causado por aproximar un proceso infinito con pasos o términos finitos.

Extrapolación de Richardson: Combinación de resultados con distintos pasos para cancelar el error principal.

Fornberg: Recurrencia estable para generar pesos de diferencias finitas.

Orden N: Mayor cantidad total de derivaciones o potencias incluida.

Pascal: Triángulo que contiene los coeficientes binomiales.

Plantilla o stencil: Conjunto de puntos vecinos usados por una diferencia finita.

Polinomio: Suma de potencias sencillas de las variables.

Precisión arbitraria: Uso de tantos dígitos internos como se soliciten.

Taylor: Polinomio que reproduce información local de una función.

22 La receta final en diez líneas

1. Elige una función suave $f(x, y)$.
2. Elige un centro (a, b) .
3. Elige el orden total N .
4. Construye Pascal hasta la fila N .
5. Evalúa la función en puntos vecinos.
6. Convierte esas evaluaciones en derivadas finitas.
7. Usa Pascal, factoriales y derivadas para armar T_N .
8. Evalúa f y T_N en una malla.
9. Busca el contorno $f - T_N = 0$.
10. Dibuja y comprueba el error, recordando que Taylor es local.

Si entendiste esta receta, entendiste el proyecto.

23 Referencias y siguiente paso

- B. Fornberg, *Generation of Finite Difference Formulas on Arbitrarily Spaced Grids*, Mathematics of Computation 51 (1988), 699–706. DOI: 10.1090/S0025-5718-1988-0935077-0.
- Documentación oficial de mpmath, sección de diferenciación numérica: mpmath.org/doc/current/mpmath.pdf.
- Documentación del proyecto y ejemplos de ejecución: github.com/Sekinal/taylor.



Siguiente experimento sugerido

Ejecuta la misma función con los tres niveles, primero en un rango pequeño y luego en uno grande. Compara derivadas, incertidumbres, RMSE e intersecciones. Ver las diferencias convierte todas estas fórmulas en intuición.